

Versi 2.0
September 2026



PEMROGRAMAN WEB

MODUL 6 - INTEGRASI FRONTEND, DOKUMENTASI & DEPLOYMENT

Author:

Christian Sri Kusuma Aditya, S.Kom., M.Kom

Nisrina Nurhafizhah

Shofwan Alif Alfani

LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	1
PENDAHULUAN.....	4
OBJECTIVES.....	4
TARGET MODUL.....	4
PERSIAPAN.....	4
KATA KUNCI.....	4
MATERI.....	5
1. Integrasi Dasar ke Laravel API.....	5
1.1. Base URL API via Environment Variable.....	5
1.2. Menampilkan Data (GET).....	6
1.3. Mengirim Data (POST) dan Menangani Error Validasi.....	7
2. Autentikasi di Frontend.....	8
2.1. Login dan Menyimpan Token.....	8
2.2. Melampirkan Token ke Protected Route.....	10
2.3. Logout.....	11
3. File Upload di Frontend.....	11
3.1. Upload File Saat Menambah Data (POST).....	11
3.2. Update Data dengan File (PATCH).....	13
3.3. Menampilkan File yang Sudah Diupload.....	14
4. Dokumentasi Project.....	14
4.1. README Project.....	14
4.2. .env.example.....	15
4.3. Dokumentasi Endpoint API.....	16
5. Deploy Readiness & Deployment.....	17
5.1. Deploy Readiness Checklist.....	17
5.2. Deploy Laravel API (Backend).....	18
5.2.1. Push Project dan Hubungkan ke Railway.....	18
5.2.2. Tambahkan Database.....	18
5.2.3. Atur Environment Variable.....	19
5.2.4. Jalankan Migration dan Storage Link.....	20
5.2.5. Konfigurasi Persistent Storage (Volume).....	21
5.2.6. Konfigurasi Logging.....	21
5.2.7. Verifikasi Deploy.....	22
5.3. Deploy Frontend (Static).....	22
5.3.1. Push dan Hubungkan Repository.....	22
5.3.2. Konfigurasi Build.....	22
5.3.3. Deploy.....	23
5.3.4. Sinkronisasi dengan Backend.....	23
5.3.5. Continuous Deployment.....	23



CODELAB.....	24
Integrasi Frontend ke REST API Perpustakaan.....	24
Setup Awal.....	24
Ketentuan.....	24
Perhatian.....	24
Output yang Diharapkan.....	24
DEMO.....	25
Ketentuan Pengerjaan.....	25
Alur yang Diharapkan.....	25
Nilai Tambah.....	26
RUBRIK PENILAIAN.....	27



PENDAHULUAN

OBJECTIVES

1. Mahasiswa mampu mengintegrasikan frontend berbasis JavaScript murni dengan REST API Laravel yang telah dibangun pada Modul 4 dan 5.
2. Mahasiswa mampu mengimplementasikan alur autentikasi berbasis JWT dari sisi klien, termasuk menyimpan, melampirkan, dan menghapus token.
3. Mahasiswa mampu menangani upload dan tampilan file dari frontend ke endpoint Laravel yang mendukung upload file.
4. Mahasiswa mampu mendokumentasikan proyek (README, .env.example, dokumentasi endpoint) dan mendeploy aplikasi full-stack ke platform hosting gratis (Railway dan Vercel).

TARGET MODUL

1. Mahasiswa mampu menghubungkan operasi CRUD (GET, POST, PATCH) dari frontend ke API Laravel menggunakan `fetch`, termasuk menangani loading state dan error validasi (status 422).
2. Mahasiswa mampu mengelola autentikasi berbasis token (login, protected route, logout) serta menerapkan konfigurasi CORS yang benar antara frontend dan backend.
3. Mahasiswa mampu mendeploy backend Laravel ke Railway dan frontend statis ke Vercel, lengkap dengan konfigurasi environment variable, database, storage, dan logging di environment production.

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum memulai praktikum.
 رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop atau personal computer.
3. Project Laravel dari Modul 4 dan 5 yang sudah berjalan (CRUD, autentikasi JWT, upload file).
4. Code editor (VS Code / sejenis) dan browser dengan DevTools untuk debugging fetch dan CORS.
5. Akun GitHub, Railway, dan Vercel (gratis) untuk keperluan Bab 5.
6. Strong motivation and commitment to learning.

KATA KUNCI

REST API Integration, JWT Authentication, CORS, File Upload, Environment Variable, Deployment (Railway & Vercel)



MATERI

1. Integrasi Dasar ke Laravel API

Backend sudah lengkap sejak Modul 4 (CRUD) dan Modul 5 (auth, middleware, dan file upload). Bagian ini menghubungkan semuanya ke frontend menggunakan JavaScript murni yang sudah dipelajari di Modul 2; fokusnya bukan mengulang sintaks fetch, melainkan menerapkannya pada API Laravel milik sendiri untuk pertama kalinya.

1.1. Base URL API via Environment Variable

Menuliskan alamat API secara langsung (hardcode) di setiap file JavaScript menyulitkan saat API pindah alamat, misalnya dari local ke production. Simpan base URL sebagai satu variabel terpusat di awal file:

```
const API_BASE_URL = 'http://localhost:8000/api';
```

Gambar 1.1: Contoh pendefinisian base URL API secara terpusat pada frontend

Jika frontend menggunakan build tool seperti Vite, base URL ini sebaiknya dipindahkan ke file `.env` milik frontend:

```
VITE_API_BASE_URL=http://localhost:8000/api
```

Gambar 1.2: Contoh konfigurasi base URL API melalui environment variable Vite

Lalu dipanggil melalui `import.meta.env.VITE_API_BASE_URL`. Dengan begini, saat aplikasi di-deploy nanti, cukup ubah nilai variabel ini tanpa menyentuh kode JavaScript lainnya.



1.2. Menampilkan Data (GET)

Contoh berikut mengambil katalog buku dari `GET /api/books`, route ini public, tidak membutuhkan token:

```

async function getBooks() {
  const container = document.querySelector('#book-list');
  container.innerHTML = 'Memuat data...';

  try {
    const response = await fetch(`${API_BASE_URL}/books`);
    const result = await response.json();

    container.innerHTML = result.data
      .map(book => `<div>${book.title} – ${book.author}</div>`)
      .join('');
  } catch (error) {
    container.innerHTML = 'Gagal memuat data buku. Periksa koneksi atau server.';
  }
}

```

Gambar 1.3: Contoh fetch GET `/api/books` dengan loading state dan error handling

Loading state ditampilkan sebelum data diterima, dan pesan error ditampilkan jika fetch gagal (misalnya server mati atau tidak ada koneksi). Dua hal ini wajib ada agar pengguna tetap mendapatkan informasi yang jelas.



1.3. Mengirim Data (POST) dan Menangani Error Validasi

Menambahkan buku baru mengirim data ke `POST /api/books`. Karena endpoint ini menggunakan Form Request Validation (Modul 4), Laravel akan mengembalikan status `422` beserta pesan error untuk setiap field jika input tidak valid. Response ini perlu ditangkap dan ditampilkan, bukan dibiarkan gagal diam-diam:

```

async function submitBook(event) {
  event.preventDefault();
  const formData = new FormData(event.target);

  const response = await fetch(`${API_BASE_URL}/books`, {
    method: 'POST',
    body: formData,
  });

  if (response.status === 422) {
    const result = await response.json();
    Object.entries(result.errors).forEach(([field, messages]) => {
      const errorEl = document.querySelector(`#error-${field}`);
      if (errorEl) errorEl.textContent = messages[0];
    });
    return;
  }

  if (!response.ok) {
    alert('Terjadi kesalahan pada server.');
```

Gambar 1.4: Contoh fetch POST `/api/books` dengan penanganan error validasi 422

Status `201` menandakan buku berhasil dibuat (sesuai konsep HTTP status code dari Modul 2), sehingga setelah berhasil, form dikosongkan dan daftar buku dimuat ulang.



Catatan: CORS

Jika muncul error “**blocked by CORS policy**” di console browser saat fetch dijalankan, konsepnya sudah dibahas secara tuntas pada Modul 5. Langkah cek cepatnya:

- Buka `config/cors.php` di project Laravel
- Pastikan `allowed_origins` mencakup alamat frontend yang dipakai (misalnya `http://127.0.0.1:5500` untuk Live Server, atau `http://localhost:5173` untuk Vite)
- Jalankan `php artisan config:clear` jika perubahan belum terlihat efeknya

2. Autentikasi di Frontend

Backend auth berbasis JWT sudah dibangun sepenuhnya pada Modul 5. Endpoint register, login, me, dan logout sudah tersedia dan sudah diuji menggunakan Postman. Bagian ini menghubungkan alur yang sama dari sisi frontend menggunakan JavaScript.

2.1. Login dan Menyimpan Token

Sesuai proses login yang sudah dibangun di Modul 5, respons yang dikembalikan berisi data pengguna beserta token JWT. Dari sisi frontend, token ini perlu disimpan agar bisa dipakai kembali di request-request berikutnya:

```
async function login(event) {
  event.preventDefault();
  const formData = new FormData(event.target);

  const response = await fetch(`${API_BASE_URL}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      email: formData.get('email'),
      password: formData.get('password'),
    }),
  });

  if (response.status === 401) {
    document.querySelector('#login-error').textContent = 'Email atau password salah.';
    return;
  }

  const result = await response.json();
  localStorage.setItem('token', result.data.token);
  window.location.href = '/dashboard.html';
}
```

Gambar 2.1: Contoh proses login dan penyimpanan token JWT ke localStorage



Token disimpan di `localStorage` agar tetap tersedia meskipun halaman di-refresh, sampai pengguna logout atau menghapus data browser secara manual.

Catatan penting

`localStorage` bisa dibaca oleh JavaScript apa pun yang berjalan di halaman tersebut, termasuk skrip pihak ketiga atau skrip jahat yang berhasil disisipkan melalui celah XSS.

Skenario risikonya konkret untuk FlipPlay: jika ada satu saja celah XSS di rendering konten Creator Studio (misalnya judul media yang di-upload user creator ditampilkan ke user lain lewat `innerHTML` tanpa sanitasi), skrip jahat yang tersisip di judul tersebut bisa membaca `localStorage.getItem('token')` milik user lain yang membuka halaman itu, lalu mengirim token tersebut ke server milik penyerang. Token yang dicuri ini dapat digunakan untuk melakukan takeover akun. Penyerang dapat menyamar sebagai user tersebut di semua rute yang dilindungi.

Alternatif yang lebih aman (di luar cakupan wajib modul ini, tapi penting diketahui sebagai trade-off):

- HTTP-only cookie: token disimpan dalam cookie dengan flag `HttpOnly`, yang membuatnya tidak bisa dibaca sama sekali oleh JavaScript. Server mengatur cookie ini melalui header respons, dan browser otomatis menyertakannya pada setiap permintaan tanpa perlu ditulis secara manual dalam JavaScript.
- Pola hybrid: access token (masa berlaku pendek) disimpan dalam variabel JavaScript biasa (di memory, hilang saat refresh), sedangkan refresh token (untuk mendapatkan access token baru) disimpan dalam cookie.

Untuk level modul ini, menyimpan token di `localStorage` masih bisa dipakai sebagai penyederhanaan pembelajaran, dengan catatan: pastikan tidak ada satu pun tempat di frontend yang merender konten user-generated (judul, deskripsi, komentar) menggunakan `innerHTML` tanpa `textContent` atau sanitasi.



2.2. Melampirkan Token ke Protected Route

Endpoint seperti `POST /api/members/{id}/borrow-book` dan `PATCH /api/members/{id}/return-book/{book_id}` dilindungi oleh authentication middleware (Modul 5), sehingga setiap request ke sana wajib menyertakan token lewat header `Authorization: Bearer <token>`:

```
async function borrowBook(memberId, bookId) {
  const token = localStorage.getItem('token');

  const response = await fetch(`${API_BASE_URL}/members/${memberId}/borrow-book`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`,
    },
    body: JSON.stringify({ book_id: bookId }),
  });

  if (response.status === 401) {
    alert('Sesi login sudah berakhir, silakan login kembali.');
```

Gambar 2.2: Contoh melampirkan token ke header Authorization pada protected route

Pola yang sama berlaku untuk `return-book`, `GET /api/auth/me` (menampilkan profil), maupun endpoint protected lainnya. Cukup ganti method, endpoint, dan body sesuai kebutuhan; header Authorization tetap sama.



2.3. Logout

Logout memanggil `POST /api/auth/logout` (juga protected, butuh token) untuk mematikan token di sisi server, lalu menghapus token dari sisi client:

```
async function logout() {
  const token = localStorage.getItem('token');

  await fetch(`${API_BASE_URL}/auth/logout`, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${token}` },
  });

  localStorage.removeItem('token');
  window.location.href = '/login.html';
}
```

Gambar 2.3: Contoh proses logout (memanggil endpoint logout dan menghapus token)

Menghapus token dari `localStorage` saja tidak cukup; tanpa memanggil endpoint logout, token lama secara teknis masih valid di server sampai masa berlakunya habis.

3. File Upload di Frontend

Backend sudah mendukung upload file sejak Modul 5. Bagian ini menghubungkan form upload di frontend ke endpoint tersebut.

3.1. Upload File Saat Menambah Data (POST)

Input file tidak bisa dikirim sebagai JSON biasa, harus menggunakan `FormData`, yang otomatis mengatur header `Content-Type` menjadi `multipart/form-data`:

```
<form id="book-form">
  <input type="text" name="title" placeholder="Judul buku" required>
  <input type="text" name="author" placeholder="Penulis" required>
  <input type="file" name="file_path" accept="image/*">
  <button type="submit">Simpan</button>
</form>
```

Gambar 3.1: Contoh penggunaan `FormData` untuk input file



```

async function submitBook(event) {
  event.preventDefault();
  const formData = new FormData(event.target);

  const response = await fetch(`${API_BASE_URL}/books`, {
    method: 'POST',
    body: formData,
  });

  if (response.status === 422) {
    const result = await response.json();
    Object.entries(result.errors).forEach(([field, messages]) => {
      const errorEl = document.querySelector(`#error-${field}`);
      if (errorEl) errorEl.textContent = messages[0];
    });
    return;
  }

  event.target.reset();
  getBooks();
}

```

Gambar 3.2: Contoh lengkap request POST dengan FormData untuk upload file

Catatan: saat menggunakan `FormData`, jangan menambahkan header `Content-Type` secara manual; browser akan otomatis mengisinya lengkap dengan `boundary` yang dibutuhkan untuk memisahkan setiap field, termasuk file-nya.



3.2. Update Data dengan File (PATCH)

Modul 5 menyebutkan bahwa implementasi upload juga perlu berjalan di method update, bukan hanya saat data pertama kali dibuat. Kendalanya, PHP secara native tidak mem-parsing body `multipart/form-data` untuk request dengan method PATCH atau PUT, variabel `$_POST` (yang menjadi dasar `$request->all()` di Laravel) hanya otomatis terisi untuk method POST. Solusi paling stabil adalah tetap mengirim request sebagai POST, tapi menyisipkan field tersembunyi `_method` agar Laravel membacanya sebagai PATCH (method spoofing, fitur bawaan Laravel):

```

async function updateBook(event, bookId) {
  event.preventDefault();
  const formData = new FormData(event.target);
  formData.append('_method', 'PATCH');

  const response = await fetch(`${API_BASE_URL}/books/${bookId}`, {
    method: 'POST',
    body: formData,
  });

  if (response.status === 422) {
    const result = await response.json();
    Object.entries(result.errors).forEach(([field, messages]) => {
      const errorEl = document.querySelector(`#error-${field}`);
      if (errorEl) errorEl.textContent = messages[0];
    });
    return;
  }

  getBooks();
}

```

Gambar 3.3: Contoh method spoofing (field method) untuk update data dengan file via PATCH

Dengan pola ini, field `file_path` bersifat opsional saat update; jika pengguna tidak memilih file baru, Laravel akan mempertahankan file lama yang sudah tersimpan (sesuai penyesuaian `StoreBookRequest` yang sudah dibuat di Modul 5 untuk membedakan aturan POST dan PATCH).



3.3. Menampilkan File yang Sudah Diupload

`BookResource` sudah mengembalikan `file_url`, sehingga file bisa langsung ditampilkan tanpa perlu menyusun path secara manual di frontend:

```
function renderBook(book) {
  return `
    <div>
      
      <p>${book.title} – ${book.author}</p>
    </div>
  `;
}
```

Gambar 3.4: Contoh function untuk merender data buku dan file gambar dari response API

4. Dokumentasi Project

Setelah aplikasi berfungsi penuh dari frontend hingga database, langkah selanjutnya yang tidak kalah penting adalah mendokumentasikan proyek agar mudah dipahami, dijalankan ulang, atau dilanjutkan oleh orang lain maupun diri sendiri di kemudian hari. Project yang berfungsi dengan baik namun tidak terdokumentasi akan sulit digunakan kembali, terutama saat bekerja dalam tim atau saat dinilai oleh asisten.

4.1. README Project

README adalah file dokumentasi utama sebuah proyek yang diletakkan di root folder dengan nama `README.md`. File ini menjadi hal pertama yang dibaca orang lain saat membuka project, sehingga perlu memuat:

Tabel 4.1: Struktur isi README project

Bagian	Isi
Deskripsi project	Penjelasan singkat aplikasi dan fungsinya
Teknologi yang digunakan	Laravel, MySQL, JavaScript, JWT, dan tools lainnya
Cara instalasi	Langkah menjalankan project dari awal (clone, install dependency, migrate)
Cara menjalankan	Perintah untuk menjalankan server backend dan frontend
Struktur folder	Penjelasan singkat folder-folder penting dalam project



Contoh potongan README untuk project sistem perpustakaan yang sudah dibangun sejak Modul 4:

```
## Instalasi

1. Clone repository
2. composer install
3. cp .env.example .env
4. php artisan key:generate
5. php artisan jwt:secret
6. php artisan migrate --seed
7. php artisan storage:link
8. php artisan serve

## Menjalankan Frontend
Buka file index.html menggunakan Live Server, atau jalankan npm run dev jika menggunakan Vite.
```

Gambar 4.1: Contoh potongan README project sistem perpustakaan

4.2. .env.example

File `.env` berisi konfigurasi environment yang bersifat sensitif (password database, JWT_SECRET), sehingga tidak boleh diunggah ke repository publik. Sebagai gantinya, buat file `.env.example` yang berisi struktur konfigurasi yang sama tanpa nilai sensitif, sebagai panduan bagi siapa pun yang menjalankan project:

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=
DB_USERNAME=
DB_PASSWORD=

JWT_SECRET=
```

Gambar 4.2: Contoh isi file `.env.example`

Dengan adanya `.env.example`, orang lain cukup menyalin file ini menjadi `.env` dan mengisi nilai sesuai konfigurasi miliknya sendiri, tanpa perlu menebak-nebak konfigurasi apa saja yang dibutuhkan. Pastikan `.env` sudah masuk ke `.gitignore` agar tidak ikut ter-commit.



4.3. Dokumentasi Endpoint API

Seluruh endpoint yang sudah dibangun sejak Modul 4 dan 5 perlu didokumentasikan agar mudah digunakan tanpa harus membuka kode Controller satu per satu:

Tabel 4.2: Dokumentasi endpoint API untuk seluruh endpoint sistem perpustakaan

Method	Endpoint	Auth	Deskripsi
GET	/api/books	Tidak	Menampilkan daftar buku
POST	/api/books	Tidak	Menambahkan buku baru (mendukung upload file)
PATCH	/api/books/{id}	Tidak	Mengubah data buku
POST	/api/auth/register	Tidak	Mendaftarkan akun baru
POST	/api/auth/login	Tidak	Login, mengembalikan token JWT
GET	/api/auth/me	Ya (Bearer)	Menampilkan profil pengguna aktif
POST	/api/auth/logout	Ya (Bearer)	Mengakhiri sesi login
POST	/api/members/{id}/borrow-book	Ya (Bearer)	Meminjam buku
PATCH	/api/members/{id}/return-book/{book_id}	Ya (Bearer)	Mengembalikan buku
DELETE	/api/books/{id}	Tidak	Menghapus data buku

Dokumentasi seperti ini bisa diletakkan langsung di README atau dipisah ke dalam file `API.md`, tergantung pada kompleksitas proyek. Yang terpenting, format dan gaya penulisannya konsisten sehingga mudah dibaca siapa pun, termasuk kolom "Auth" yang menandakan endpoint mana yang membutuhkan token Bearer. Ini langsung membantu siapa pun yang membaca untuk mengetahui endpoint mana yang perlu login terlebih dahulu sebelum diakses.



5. Deploy Readiness & Deployment

Setelah aplikasi berfungsi penuh dan terdokumentasi, tahap akhir sebelum proyek dinilai adalah memastikan aplikasi siap dijalankan di server sungguhan (production), lalu benar-benar di-deploy.

5.1. Deploy Readiness Checklist

Sebelum deploy, pastikan hal-hal berikut agar aplikasi berjalan aman dan stabil di environment production:

Tabel 5.1: Deploy Readiness Checklist

Aspek	Yang Perlu Dipastikan
Environment	<code>APP_ENV=production</code> , <code>APP_DEBUG=false</code> agar detail error tidak terekspos ke publik
APP_KEY & JWT_SECRET	Sudah digenerate ulang khusus untuk environment production, jangan pakai kunci yang sama dengan local
Database	Kredensial database production sudah dikonfigurasi di <code>.env</code> , terpisah dari database local
Migration & Seeder	Seluruh migration sudah dijalankan (<code>php artisan migrate</code>) di database production
Storage	<code>php artisan storage:link</code> sudah dijalankan di server production agar file upload tetap bisa diakses
CORS	<code>allowed_origins</code> di <code>config/cors.php</code> sudah diisi alamat frontend production, bukan lagi <code>localhost</code>
Build frontend	Base URL API di environment variable frontend sudah mengarah ke alamat backend production
.env	File <code>.env</code> tidak diikutsertakan ke repository publik (dipastikan masuk <code>.gitignore</code>)

Catatan penting: `APP_DEBUG=false` sangat krusial; kalau bernilai `true`, error di production akan menampilkan detail kode dan struktur aplikasi ke publik, termasuk kemungkinan membocorkan `JWT_SECRET` lewat stack trace.

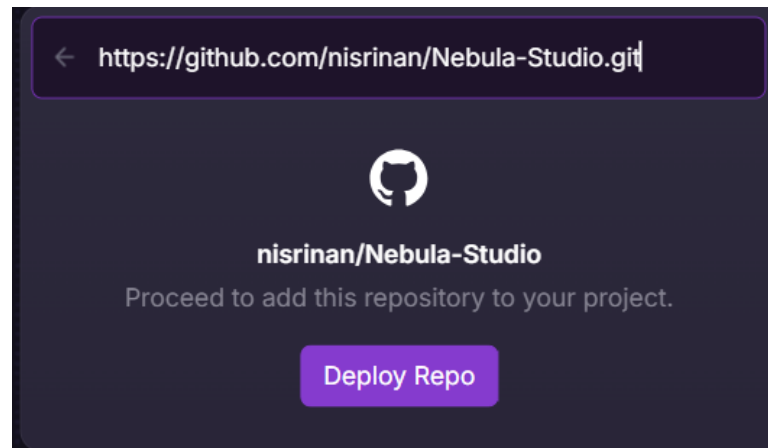


5.2. Deploy Laravel API (Backend)

Backend Laravel di-deploy menggunakan platform gratis seperti Railway. Alur umumnya:

5.2.1. Push Project dan Hubungkan ke Railway

1. Push project Laravel ke repository GitHub. Pastikan file `.env` tidak ikut ter-commit, cukup `.env.example` sebagai referensi daftar variabel yang dibutuhkan.
2. Login ke railway.app, klik New Project, pilih Deploy from GitHub repo, lalu hubungkan repository yang sudah dipush.
3. Railway akan otomatis mendeteksi project Laravel dan mulai proses build pertama (biasanya akan gagal di tahap ini karena environment variable belum diisi; itu normal, lanjut ke langkah berikutnya).



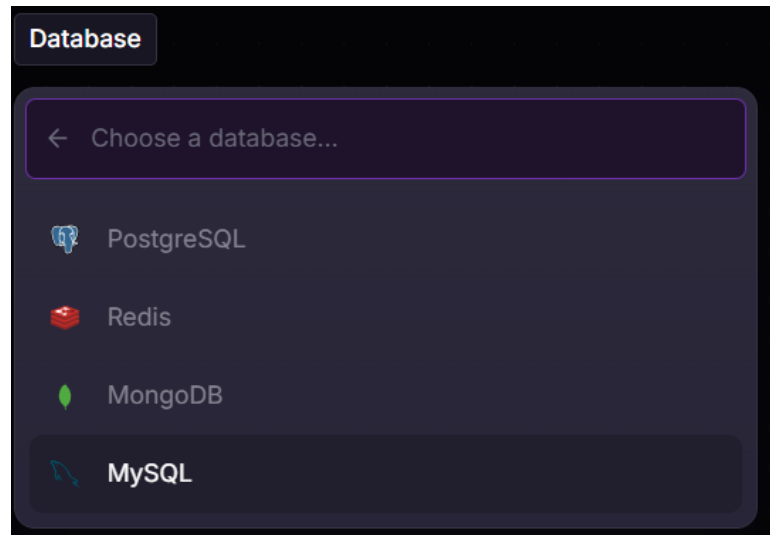
Gambar 5.1: Tampilan proses build pertama project Laravel di Railway

Catatan: jika backend dan frontend digabungkan dalam satu repository yang sama (monorepo), Railway secara default akan mencoba build dari root repo. Buka tab Settings pada service yang baru dibuat, cari bagian Root Directory, lalu isi dengan path folder backend Laravel (misalnya `/backend` atau `/api`). Tanpa langkah ini, Railway bisa gagal mendeteksi project Laravel, atau malah salah mendeteksi folder frontend sebagai root build.

5.2.2. Tambahkan Database

1. Di dalam project yang sama, klik **New** (pojok kanan atas canvas) atau tekan `Ctrl+K` lalu ketik `database`.
2. Pilih jenis database yang digunakan, misalnya **PostgreSQL** atau **MySQL**.





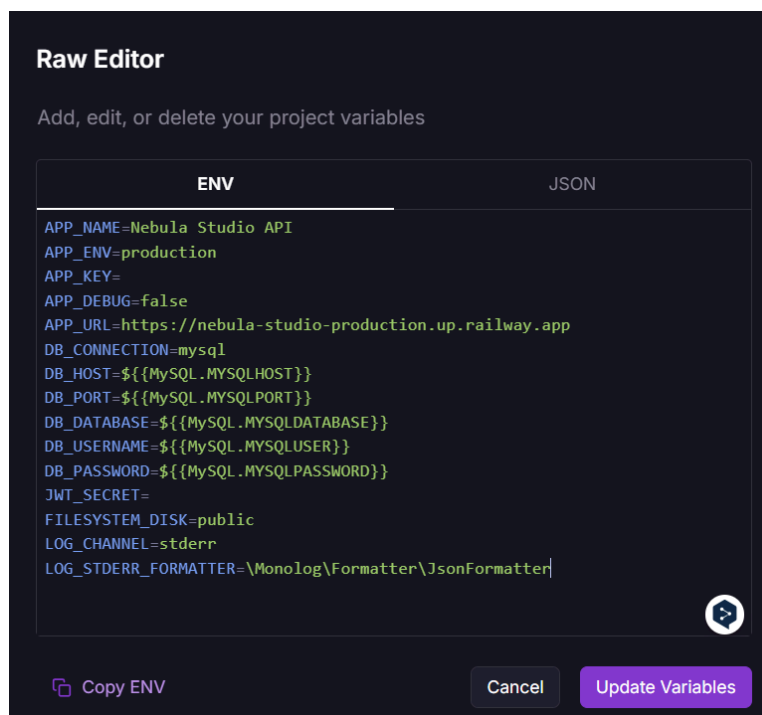
Gambar 5.2: Tampilan pemilihan jenis database (PostgreSQL/MySQL) di Railway

3. Railway otomatis membuat service database terpisah beserta variabel koneksinya (`DATABASE_URL`, `MYSQLHOST`, dst).
4. Di service Laravel, buka tab **Variables**, referensikan variabel koneksi ke database tersebut, contoh: `DB_HOST={{MySQL.MYSQLHOST}}`,
`DB_PORT={{MySQL.MYSQLPORT}}`, `DB_DATABASE={{MySQL.MYSQLDATABASE}}`,
`DB_USERNAME={{MySQL.MYSQLUSER}}`,
`DB_PASSWORD={{MySQL.MYSQLPASSWORD}}`.

5.2.3. Atur Environment Variable

1. Klik tile service Laravel, buka tab **Variables**.
2. Isi minimal variabel berikut sesuai isi `.env.example`:
 - a. `APP_KEY` (generate dulu di lokal dengan `php artisan key:generate --show`, lalu copy hasilnya)
 - b. `APP_ENV=production`
 - c. `APP_DEBUG=false`
 - d. `JWT_SECRET`
 - e. `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`
3. Untuk mempercepat, gunakan tombol **RAW Editor** agar bisa paste semua variabel sekaligus dalam format `.env`, daripada menginput satu per satu.
4. Tambahkan juga dua variabel berikut agar log aplikasi bisa terbaca di dashboard Railway (dibahas detail di 5.2.5):





Gambar 5.3: Contoh dua variabel logging

5. Save. Railway akan secara otomatis memicu redeploy setiap kali variabel diubah.

5.2.4. Jalankan Migration dan Storage Link

Railway menyediakan akses shell langsung ke dalam container yang sedang berjalan melalui menu Shell pada service (tidak perlu menginstal Railway CLI untuk cara ini). Setelah masuk ke shell tersebut, jalankan langsung:

```
root@ad479689191f:/app# php artisan migrate --force
```

Gambar 5.4: Perintah migration dengan flag `--force` di shell Railway

```
root@ad479689191f:/app# php artisan storage:link
```

Gambar 5.5: Perintah `storage:link` di shell Railway

Flag `--force` wajib disertakan karena Laravel akan menolak menjalankan migration di `APP_ENV=production` tanpa flag ini, sebagai pengaman agar migration tidak disengaja dijalankan di server live.

Catatan: perintah `railway run` dan `railway shell` adalah command yang dijalankan dari terminal lokal di komputer sendiri (menyuntikkan environment variable Railway ke proses lokal), bukan perintah yang dijalankan di dalam shell container itu sendiri. Kalau kamu sudah berada di dalam shell container (ditandai dengan prompt



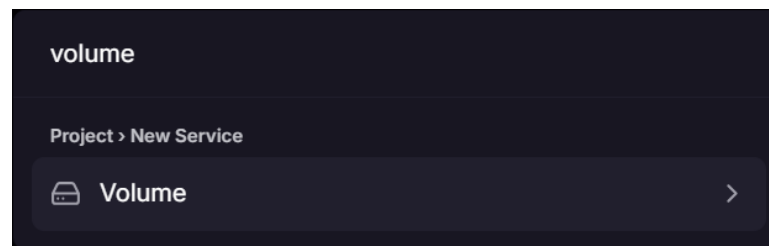
seperti `root@<container-id>:/app#`), jalankan langsung perintah Artisan-nya tanpa prefix `"railway run"`.

5.2.5. Konfigurasi Persistent Storage (Volume)

Container Railway bersifat *ephemeral* — pada setiap redeploy, container lama dihapus sepenuhnya beserta seluruh file yang ditulis ke disk. Jika API menyimpan file upload (misalnya gambar profil) ke `storage/app/public`, file tersebut akan hilang setiap kali redeploy, kecuali jika disimpan di Volume (block storage persisten yang terpisah dari container).

Cara menambahkan volume:

1. Klik tile service Laravel, buka tab **Settings**.
2. Scroll ke bagian **Volumes**, lalu klik **Add Volume** (jika opsi ini tidak muncul, gunakan `Ctrl+K` lalu ketik `volume`, atau klik kanan langsung pada tile service tersebut).



Gambar 5.6: Tampilan opsi Add Volume pada Settings service Laravel di Railway

3. Isi **Mount Path:** `/app/storage/app/public`.
4. Klik **Add** — Railway otomatis redeploy service tersebut.
5. Jalankan ulang `php artisan storage:link` setelah volume terpasang, karena symlink `public/storage` perlu dibentuk ulang agar file di volume dapat diakses melalui URL publik.

5.2.6. Konfigurasi Logging

Secara default, Laravel menulis log ke file `storage/logs/laravel.log`, yang tidak bisa dibaca melalui dashboard Railway karena filesystem-nya bersifat ephemeral. Variabel `LOG_CHANNEL=stderr` dan `LOG_STDERR_FORMATTER=\Monolog\Formatter\JsonFormatter` (sudah diisi di langkah 5.2.3) mengarahkan log Laravel ke stream `stderr`, sehingga bisa dibaca langsung lewat tab Deployments → View Logs pada service tersebut secara real-time, dalam format JSON.



5.2.7. Verifikasi Deploy

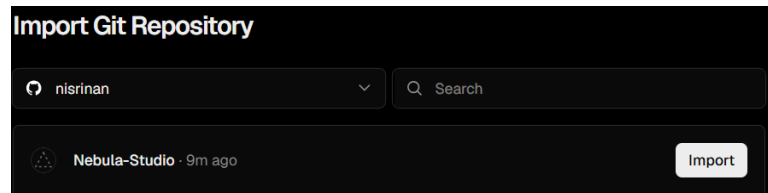
Setelah semua langkah di atas selesai, Railway memberikan domain publik otomatis dengan format `*.up.railway.app` yang tercantum di tab Settings → Networking. Akses domain tersebut untuk memastikan API merespons, lalu cek tab Deployments → View Logs untuk memastikan tidak ada error saat boot aplikasi.

5.3. Deploy Frontend (Static)

Untuk frontend berupa HTML, CSS, dan JavaScript statis, Vercel adalah pilihan yang sederhana dan gratis:

5.3.1. Push dan Hubungkan Repository

1. Push folder frontend ke repository GitHub. Pastikan `index.html` berada di root folder project. Jika file statis berada di subfolder (misalnya `dist` atau `build`), catat nama foldernya untuk diisi pada langkah konfigurasi.
2. Login ke vercel.com, klik Add New → Project.
3. Pilih Import Git Repository, lalu hubungkan repository frontend tersebut.

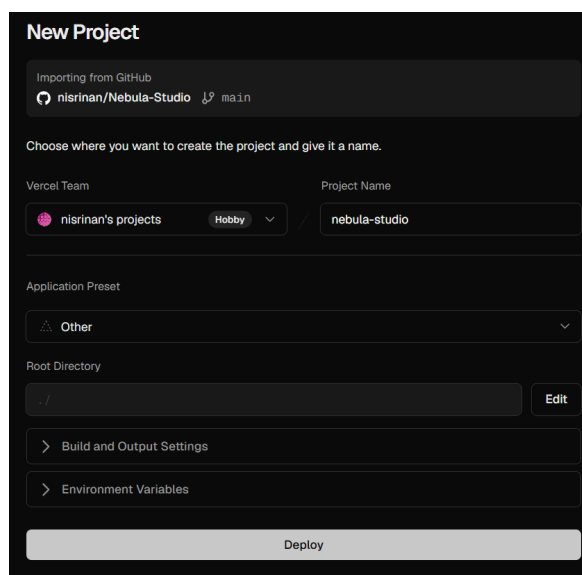


Gambar 5.7: Tampilan opsi Import Git Repository di Vercel

5.3.2. Konfigurasi Build

1. Pada halaman konfigurasi sebelum deploy, pilih Framework Preset: Other jika project murni statis tanpa framework atau bundler.
2. Kosongkan Build Command jika tidak ada proses build (HTML/CSS/JS polos).
3. Isi Output Directory sesuai lokasi file statis kamu (default `./` jika `index.html` di root, atau nama subfolder seperti `dist` jika menggunakan bundler).

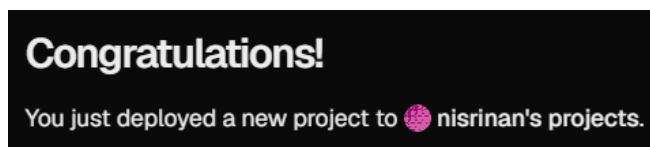




Gambar 5.8: Tampilan konfigurasi build di Vercel

5.3.3. Deploy

Klik **Deploy**. Vercel akan clone repository, menjalankan build (jika dikonfigurasi), lalu menerbitkan URL publik dengan domain `*.vercel.app` beserta HTTPS otomatis.



Gambar 5.9: Tampilan hasil deploy dengan domain `.vercel.app`

5.3.4. Sinkronisasi dengan Backend

Setelah frontend live, dua hal berikut wajib dilakukan agar integrasi tidak putus:

1. Update environment variable base URL API (dibahas di Bab 1) agar mengarah ke domain `*.up.railway.app` dari backend production, bukan lagi `localhost`.
2. Update konfigurasi CORS di backend Laravel (`config/cors.php`, bagian `allowed_origins`) agar mencakup domain `*.vercel.app` dari frontend. Tanpa langkah ini, integrasi yang berjalan mulus di lokal akan langsung terblokir CORS begitu online, karena browser menolak request cross-origin dari domain yang tidak diizinkan.

5.3.5. Continuous Deployment

Setiap push baru ke branch utama GitHub akan otomatis memicu redeploy di kedua platform (Railway dan Vercel) tanpa perlu langkah manual tambahan; cukup `git push`, dan kedua service akan mengambil kode terbaru serta menerbitkan versi baru secara otomatis.



CODELAB

Integrasi Frontend ke REST API Perpustakaan

Lanjutkan repository Laravel perpustakaan dari Codelab Modul 4-5. Buat frontend JavaScript murni yang terhubung ke backend tersebut.

Setup Awal

1. Pastikan backend perpustakaan (Modul 4-5) berjalan (`php artisan serve`).
2. Siapkan folder frontend baru (HTML/CSS/JS murni, atau Vite jika ingin base URL melalui `.env`).
3. Pastikan `allowed_origins` di `config/cors.php` sudah mencakup alamat frontend yang digunakan.

Ketentuan

- Tampilkan katalog buku dari `GET /api/books`, lengkap dengan loading state dan pesan error jika permintaan fetch gagal.
- Buat form tambah buku yang mengirim `POST` ke `/api/books`, menangani sukses (201) dan gagal validasi (422) — pesan error 422 wajib tampil per field, bukan alert generik.
- Simpan base URL API sebagai satu variabel terpusat, bukan hardcode berulang di banyak file.

Perhatian

- Cukup GET & POST
- Error "Failed to fetch" yang muncul terus biasanya disebabkan oleh CORS, bukan masalah koneksi.

Output yang Diharapkan

1. Daftar buku ditampilkan dari database asli (bukan dummy).
2. Data valid → buku baru langsung muncul tanpa reload manual.
3. Data tidak valid (misal judul kosong) → error per field tampil di form.



DEMO

Pada modul ini, API FlipPlay yang telah dilengkapi dengan autentikasi JWT, role pengguna, Creator Studio, file upload, dan CORS pada Modul 5 akan dihubungkan dengan frontend.

Frontend harus menyesuaikan tampilan berdasarkan role pengguna yang sedang login, menangani proses upload file dari sisi client, serta menampilkan loading state dan pesan error. Project juga harus didokumentasikan dengan baik agar dapat dijalankan kembali oleh orang lain.

Ketentuan Pengerjaan

Lanjutkan API FlipPlay dari Demo Modul 5. Jangan membangun ulang project dari awal. Project yang dibuat wajib memenuhi ketentuan berikut:

- Menyediakan halaman register dan login.
- Simpan token JWT dari respons API setelah login.
- Menghapus token dan memanggil endpoint logout saat pengguna keluar.
- Melampirkan token pada setiap request ke protected route.
- Sesuaikan tampilan Creator Studio berdasarkan role pengguna yang sedang login.
- Tidak menampilkan Creator Studio kepada pengguna yang tidak memiliki hak akses.
- Menyediakan form upload file menggunakan `FormData`.
- Menampilkan file yang berhasil diunggah melalui URL dari API.
- Menampilkan loading state pada setiap proses pengambilan atau pengiriman data.
- Menampilkan pesan error, termasuk error validasi.
- Menyimpan base URL API menggunakan environment variable atau konfigurasi terpusat.
- Menyediakan `README.md`.
- Menyediakan `.env.example`.
- Menyediakan dokumentasi endpoint API.
- Dokumentasi endpoint minimal memuat method, endpoint, Auth, Role, dan deskripsi.
- Menyertakan Postman Collection untuk pengujian endpoint API.

Alur yang Diharapkan

- Pengguna melakukan registrasi dan login sebagai pengguna baru dengan role `user`.
- Token berhasil disimpan dan halaman utama menampilkan konten FlipPlay.
- Creator Studio tidak ditampilkan kepada pengguna dengan role `user`.
- Pengguna dengan role `creator` melakukan login.
- Menu Creator Studio ditampilkan setelah login berhasil.
- Creator menambahkan konten baru melalui formulir.
- Creator mengunggah file cover atau gambar menggunakan `FormData`.
- Konten yang baru diunggah langsung tampil di halaman utama.



- File yang diunggah dapat diakses melalui URL dari API.
- Pengguna dengan role **user** tidak dapat mengakses Creator Studio.
- Pengguna yang belum login juga tidak dapat mengakses Creator Studio.
- Pengguna melakukan logout.
- Setelah logout, pengguna mencoba melakukan aksi protected dan request ditolak.

Nilai Tambah

Deployment bersifat **opsional** dan dapat digunakan sebagai nilai tambah.

Mahasiswa yang melakukan deployment dapat memperoleh nilai tambahan apabila:

- Backend berhasil dijalankan melalui URL publik.
- Frontend berhasil dijalankan melalui URL publik.
- Frontend dapat terhubung dengan backend production.
- CORS production dikonfigurasi dengan benar.
- Project dapat dibuka dari perangkat atau browser lain.

Railway dan Vercel dapat digunakan sebagai contoh, tetapi tidak wajib. Mahasiswa dapat menggunakan platform lain, seperti Render, Fly.io, Koyeb, Netlify, Cloudflare Pages, atau platform lain yang sesuai.



RUBRIK PENILAIAN

Aspek Penilaian	Poin (%)
Codelab	Total 10%
Integrasi GET (katalog buku) dengan loading & error state	4
Integrasi POST (form tambah buku) dengan penanganan sukses & error 422 per field	4
Base URL API tersentralisasi (bukan hardcode berulang)	2
Demo	Total 80%
Autentikasi JWT di frontend (login, simpan & lampirkan token, logout)	20
Role-based UI (akses Creator Studio menyesuaikan role pengguna)	20
File upload (FormData) & tampilan file via URL publik	10
Loading state & error message pada seluruh proses fetch	10
Base URL API sebagai environment variable terpusat	10
Dokumentasi project (README, .env.example, dokumentasi endpoint termasuk Auth & Role)	10
Pemahaman	Total 10%
Ketepatan menjawab pertanyaan	10

